TECHNICAL STUDY EDITION

MIPS for Firmware Reverse Engineering and Debugger Development

A Nine-Lesson Technical Textbook and Field Reference

Independent Study Edition | October 2026 | Classic MIPS32 teaching baseline

Contents

How to Use This Book **4**

Lesson 1. Memory, Hexadecimal, and CPU Fundamentals **6**
1.1 Bits, bytes, and hexadecimal . . . 6
1.2 Signed values and two's complement . . . 6
1.3 Addresses, offsets, and dereferencing . . . 6
1.4 Endianness is about byte order, not bit order . . . 7
1.5 Alignment and instruction boundaries . . . 7
1.6 The CPU mental model . . . 7

Lesson 2. MIPS Registers and Assembly Basics **9**
2.1 What a register is . . . 9
2.2 `zero`, `s0`, `v0`, and `ra` in practice . . . 9
2.3 Why operand order cannot be memorized as one universal rule . . . 10
2.4 Register traces should be concrete . . . 10
2.5 Calling-convention thinking begins here . . . 10

Lesson 3. Loads, Stores, and Memory Access **12**
3.1 One addressing form, opposite data directions . . . 12
3.2 Width and signedness table . . . 12
3.3 Why `lbu v0,0xE(v0)` is legal and important . . . 12
3.4 Loads from C structure fields . . . 13
3.5 Following base pointers backward . . . 13
3.6 A complete miniature trace . . . 13
3.7 Unaligned access, MMIO, and practical cautions . . . 14

Lesson 4. Arithmetic, Logic, and Bit Manipulation **15**
4.1 Arithmetic is arithmetic on fixed-width bit patterns . . . 15
4.2 Bitwise logic: AND, OR, XOR, and NOT . . . 15
4.3 Shifts and their meaning . . . 16
4.4 Comparisons and predicate formation . . . 16
4.5 Reconstruct the real-world flag test . . . 16

Lesson 5. Branches, Jumps, and Execution Flow **18**
5.1 Branches make control flow conditional . . . 18
5.2 Direct jumps, indirect jumps, and links . . . 18
5.3 Delayed branches: what actually executes . . . 18
5.4 Control-flow graph reasoning . . . 19
5.5 Calls and return values . . . 19
5.6 Debugger consequences of delay slots . . . 19

Lesson 6. Functions, Stack, and Calling Conventions **21**
6.1 The stack is a memory discipline, not a magic CPU object . . . 21
6.2 O32 as a concrete convention . . . 21
6.3 A realistic function prologue and epilogue . . . 21
6.4 `ra` is not a nesting mechanism by itself . . . 22
6.5 Stack spills and local variables . . . 22
6.6 Derive argument meaning from callers . . . 22
6.7 ABI complications that matter to debuggers . . . 22

Lesson 7. Machine Code, Ghidra, and Reverse Engineering **24**
7.1 Assembly is a readable representation of bits . . . 24
7.2 Decode `lw v0,0x204(s0)` manually . . . 24
7.3 Decode a branch destination . . . 24

7.4 Start with the *question*, then choose the tool . . . 25
7.5 What an XREF does and does not mean . . . 25
7.6 A reliable backward trace . . . 26
7.7 Common decompiler and symbol traps . . . 26

Lesson 8. Exceptions, Breakpoints, and Debugger Internals 28
8.1 What a debugger must be able to do . . . 28
8.2 Exceptions and CP0 concepts . . . 28
8.3 Why EPC is not always simply “the PC that faulted” . . . 29
8.4 Software breakpoint lifecycle . . . 29
8.5 Hardware breakpoints and watchpoints . . . 29
8.6 Reading CPU context accurately . . . 30
8.7 Correct resume is an architectural action . . . 30
8.8 GDB remote protocol in context . . . 30

Lesson 9. Hooks, Patching, and the MediaTek Application 31
9.1 Patching changes executable behavior, not just bytes . . . 31
9.2 Displaced instruction relocation is the hard part . . . 31
9.3 Instruction cache and multiple processors . . . 31
9.4 The MediaTek shared-memory case: establish what is known . . . 32
9.5 Two coordinate systems, one backing memory? . . . 32
9.6 A defensible investigative decision tree . . . 32
9.7 Build debugger capabilities in safe layers . . . 33
9.8 Final guided analysis . . . 33

Appendix A. MIPS Field Reference 35
A.1 General-purpose register map . . . 35
A.2 Common load/store quick reference . . . 35
A.3 Arithmetic, logical, and control-flow quick reference . . . 36
A.4 Critical mental formulas . . . 36
A.5 Common mistakes and their corrections . . . 37

Appendix B. Checkpoint Answer Key 38
B.1 Lesson 1 . . . 38
B.2 Lesson 2 . . . 38
B.3 Lesson 3 . . . 38
B.4 Lesson 4 . . . 38
B.5 Lesson 5 . . . 38
B.6 Lesson 6 . . . 38
B.7 Lesson 7 . . . 39
B.8 Lesson 8 . . . 39
B.9 Lesson 9 . . . 39

Appendix C. Advanced MIPS Topics You Will Encounter Later 40
C.1 HI/LO multiplication, division, and hazards . . . 40
C.2 Atomic `ll/sc` and breakpoint safety . . . 40
C.3 COP1 and floating-point state . . . 40
C.4 CP0 configuration and architectural hazards . . . 40
C.5 TLB, virtual segments, and fault analysis . . . 40
C.6 microMIPS, MIPS16e, and instruction patching . . . 41
C.7 DSP, SIMD, and implementation-specific extensions . . . 41
C.8 Concurrency and interrupt-time races . . . 41
C.9 Firmware boot, relocation, and runtime memory identity . . . 41

Appendix D. Debugger Correctness Checklist 42
D.1 Architecture identity . . . 42
D.2 Memory model . . . 42

D.3 Software breakpoint invariants . . . 42
D.4 Context and protocol invariants . . . 42
D.5 Instrumentation invariants . . . 42
D.6 Suggested validation tests . . . 43

Appendix E. Worked Integrative Case Study **44**
E.1 The code and starting state . . . 44
E.2 Trace the frame . . . 44
E.3 Proving identity is a separate proof obligation . . . 44
E.4 Make a prediction under a changed state . . . 44
E.5 Your capstone exercise . . . 45

Appendix F. Glossary **46**

Appendix G. Five Detailed Tracing Laboratories **47**
G.1 The register stops being a pointer . . . 47
G.2 Signed load changes a branch . . . 47
G.3 Reconstruct a cumulative shared-memory layout . . . 47
G.4 Track argument preparation across a call . . . 48
G.5 Software-breakpoint execution-once problem . . . 48

Appendix H. Target-Identification Worksheet **50**

References and Further Reading **51**
Final reminder: what this book can and cannot certify . . . 51

How to Use This Book

This book follows the **nine-lesson, four-hour-and-fifty-minute study plan** in its original order. It is both an instructional companion and a reference for revisiting difficult ideas. The timetable is a pacing target, **not a claim that professional mastery fits into one afternoon**. If a checkpoint reveals a misconception, stop, work through the relevant examples, and return to the checkpoint before progressing.

Working assumptions for examples. Unless stated otherwise, examples use classic **MIPS32 Release 2, 32-bit general-purpose registers, little-endian data, and O32-style conventions**. These are **teaching assumptions**, *not confirmed characteristics of your particular MediaTek image*. Some MIPS revisions remove branch delay slots or alter instructions; MIPS16e and microMIPS use different encodings and instruction sizes. Hardware-debug features and coprocessor facilities also vary. Always identify the actual processor, ISA mode, endianness, ABI, and exception configuration before transferring an example to firmware.

What is specific to your investigation. Prior observations indicate a large shared-memory region whose modem-facing base is approximately `0x40000000`, a candidate subregion named `SMEM_USER_CCISM_SCP` near major-region offset `0xFA00`, and an observed screen-correlated word near `0xFC04`. The arithmetic `0xFC04 - 0xFA00 = 0x204` is exact; identifying a particular instruction as the reader or writer remains **unproved** without data-flow and address-mapping evidence. The screen-state observations show correlation, not established field semantics. The same caveat applies to the processor that writes the word.

How a study session works. For each lesson: read the stated goal; examine the worked example line by line; predict the machine state in the *Stop and predict* section; answer the checkpoint without looking ahead; then compare with the answer key at the back. A correct number with incorrect reasoning is not a pass. Revisit the topic after a break and again on a later day. This reflects instructional research on worked examples, retrieval practice, and distributed practice [E1-E2].

Notation. Hexadecimal is prefixed with `0x`; e.g., `0x20` means decimal 32. `M32[A]` means the 32-bit value read from memory at address `A` using the current endianness. `R[s0]` means the contents of register `s0`. These are teaching notations, not MIPS instructions. All addresses, flags, data layouts, and instruction snippets marked *synthetic* are examples created for learning, not claimed extracts from your firmware.

Session segment	Planned minutes	Core deliverable
Lesson 1. Memory, Hex, CPU Fundamentals	25	Explain bytes, offsets, PC, and endianness
Lesson 2. Registers and Assembly	30	Trace register changes and identify operands
Lesson 3. Loads, Stores, Structures	35	Prove effective addresses and data movement
Lesson 4. Arithmetic, Logic, Flags	30	Reconstruct bit tests and arithmetic
Lesson 5. Branches, Calls, Control Flow	35	Trace execution including delay slots
Break	5	Retrieval without notes
Lesson 6. Functions, Stack, ABI	35	Rebuild a complete function's state
Lesson 7. Encoding, Ghidra, RE	30	Decode bytes and trace provenance
Lesson 8. Exceptions and Debuggers	35	Explain a sound breakpoint lifecycle
Break	5	Retrieval without notes
Lesson 9. Patching and MediaTek Case	25	Defend a feasible instrumentation design

Source priority. For instruction-level claims use the relevant revision of the official *MIPS Architecture for Programmers* Volume II. For CP0, exception, TLB, and address-space behavior use Volume III. For calling conventions use the target ABI specification and observed compiler code. For tooling use the Ghidra and GDB manuals. References and direct URLs appear in the final section [M1-M4, T1-T3].

Lesson 1. Memory, Hexadecimal, and CPU Fundamentals

Scheduled time: 25 minutes. Mastery objective: explain exactly what a byte, address, offset, aligned word, and CPU register are; compute a load's effective address and describe how bytes form a value.

1.1 Bits, bytes, and hexadecimal

A **bit** is a binary digit: zero or one. Eight bits form a **byte**. A **nibble** is four bits. In standard MIPS terminology a **halfword** is 16 bits (2 bytes), a **word** is 32 bits (4 bytes), and a **doubleword** is 64 bits (8 bytes). A 32-bit processor need not have only 32-bit data: byte and halfword operations remain possible.

```
bit indices:  7 6 5 4 3 2 1 0
bits:         0 1 0 0 0 0 0 0
                 ^ bit 6 is set
binary 01000000 = hex 0x40 = decimal 64
```

The **bit at index 6** is the *seventh* physical bit when counted from the right starting at one. The distinction matters whenever documentation names bits by index. A set bit means `1`; whether that means enabled, disabled, pending, error, or any other concept depends on the field definition. Binary digits need not represent booleans.

Hexadecimal is base 16. Its digits are `0` through `9`, then `A` through `F` for values 10 through 15. Each hex digit represents exactly four bits: `0xF = 1111`, `0xA = 1010`. Thus `0xFF` occupies two hex digits, or eight bits, and represents decimal 255 when interpreted as unsigned. Position determines weight: `0x204 = 2*256 + 0*16 + 4 = 516` decimal. `0x100 = 256`; `0x20 = 32`; `0xFC04` is a hexadecimal number, not the decimal number 6,404.

A 32-bit word contains eight hex digits when written with leading zeros: `0x00000040`. Leading zeros do not change its unsigned numerical value, but the fixed-width form reminds you that the value occupies a 32-bit register or field. Register width, integer signedness, and in-memory byte order are separate questions.

1.2 Signed values and two's complement

An unsigned 8-bit value ranges from 0 to 255. A signed 8-bit two's-complement value ranges from -128 to 127. The same physical byte `0xFF` can be read as unsigned 255 or signed -1. The bits do not change. The **interpretation** changes. With an N-bit two's-complement number, a bit pattern whose high bit is one has signed value `unsigned_value - 2^N`. For example, `0xFFFFFFFC` in a 32-bit register is unsigned 4,294,967,292 or signed -4.

Sign extension preserves the signed value when widening a smaller signed integer. Sign-extending 8-bit `0x80` into 32 bits gives `0xFFFFFF80`, representing -128. **Zero extension** instead gives `0x00000080`, representing 128. This is why the difference between `lb` and `lbu` will later matter enormously.

1.3 Addresses, offsets, and dereferencing

On a byte-addressable processor, an **address** identifies a byte position. A **base** is the reference address from which an **offset** is measured. The effective address of a typical memory operand is:

[A_{effective} = R[base register] + sign-extended offset.]

Example: if `s0 = 0x80001000`, then `0x24(s0)` refers to address `0x80001024`. The register stores the number `0x80001000`; the offset is `0x24`; the sum is an **address**; the value stored at that address is a *different thing*. **Dereferencing** means accessing memory at an address held or computed from a pointer.

```
            base = 0x80001000
            |
            v
0x80001000  [byte]
0x80001001  [byte]
     ...
0x80001024  [byte]  <-- effective address = base + 0x24
0x80001025  [byte]
0x80001026  [byte]
0x80001027  [byte]  <-- end of four-byte word access
```

`0xFC04 - 0xFA00 = 0x204`: this calculation says an interesting location is **0x204 bytes into a sub-region beginning at major-region offset 0xFA00**. It gives that location a relationship to a meaningful region; it does *not* prove the function or meaning of the field.

1.4 Endianness is about byte order, not bit order

A 32-bit value `0x12345678` has four bytes: `12`, `34`, `56`, `78`. In **little-endian** memory, the least significant byte `78` is at the lowest address; in **big-endian** memory, `12` is at the lowest address.

```
         base+0 base+1 base+2 base+3
Little:    78     56     34     12
Big:       12     34     56     78
        (same 32-bit numeric value: 0x12345678)
```

To recover a word from a little-endian byte dump `40 00 00 00`, reverse the significance order into `0x00000040`. A byte load reads one byte, so endianness cannot reorder bytes *within* that single byte. It only affects multi-byte interpretation. Instruction encoding must be interpreted using the target's instruction-endianness and ISA mode as configured in the disassembler.

1.5 Alignment and instruction boundaries

An address is **4-byte aligned** if divisible by four: its lowest two address bits are zero. `0xFC04` is divisible by four; `0xFC02` is not. A conventional MIPS32 `lw` ordinarily requires word alignment, a halfword load requires 2-byte alignment, and byte accesses require no stronger alignment. Unaligned access may cause an address-error exception, although architecture-specific mechanisms, special instructions, or operating-system fixups can change the observed result.

Classic MIPS32 uses 32-bit instructions, ordinarily at four-byte boundaries. That rule is **not universal** across MIPS16e or microMIPS, which may encode shorter instructions. Changing the wrong number of bytes when patching code can split an instruction or its delay-slot relationship.

1.6 The CPU mental model

The CPU repeatedly obtains an instruction from code memory, interprets it using the ISA, and updates architectural state. State includes register values, the PC, relevant control registers, and memory effects. The CPU does *not* inherently understand a C variable name such as `screen_off`; it sees bit patterns, addresses, operations, and control flow. Names are an aid reconstructed by

tools and analysts.

```
Instruction at PC --> decode --> inputs from registers/memory
                                        |
                                        v
                                ALU / load / store / branch
                                        |
                                        v
                                new architectural machine state
```

The **PC** identifies the instruction stream's execution position. Sequential execution is not always simply `PC += 4`: branches, jumps, exceptions, compressed ISA modes, and delay slots alter that model. In our classic 32-bit examples, sequential instruction addresses ordinarily advance by four bytes.

Worked example: separate address, memory value, and register

Starting state, synthetic: `s0=0xA000`, bytes at `0xA020..0xA023` are `40 00 00 00`, little endian. The instruction `lw v0, 0x20(s0)` uses effective address `0xA020`; reads **four** bytes; assembles the word `0x00000040`; and writes that value to `v0`. Neither `s0` nor memory is changed. The address `0xA020` is not the loaded value.

Stop and predict

1. Convert `0x2C` into decimal, then explain why the answer is not simply 2 + 12.
2. Given `base = 0x5000`, find target address at `+0x204`.
3. Given little-endian bytes `EF BE AD DE` at four consecutive addresses, reconstruct their 32-bit word.
4. Which of `0xFC04` and `0xFC06` is word aligned, and how can you tell quickly?
5. If `0x40` changes to `0x00`, what observation is justified, and what stronger semantic conclusion is *not yet* justified?

Pass standard: compute correct addresses and values, *and* explain why address, offset, and contents are distinct. Answers are in Appendix B.

Lesson 2. MIPS Registers and Assembly Basics

Scheduled time: 30 minutes. Mastery objective: identify the roles of registers and operands, follow register transfers, and distinguish an ISA requirement from an ABI convention.

2.1 What a register is

A **register** is a CPU-visible storage location referenced by an instruction. A 32-bit GPR holds 32 bits, commonly interpreted as an integer, pointer, address component, or arbitrary flags. A register is not automatically a pointer merely because a disassembler prints `s0`; the meaning comes from **how code produces and uses its value**. The conventional MIPS32 register file contains 32 named general-purpose registers. Some have hardware-enforced behavior; many names merely describe conventions followed by compilers.

Reg no.	Names	Typical O32 ABI role	Notes
0	`zero`	constant zero	Reads zero; writes discarded
1	`at`	assembler temporary	May be used by macro expansion
2-3	`v0-v1`	return values	Also ordinary registers during computation
4-7	`a0-a3`	first 4 argument words	Caller-saved in typical O32 use
8-15	`t0-t7`	temporaries	Caller-saved
16-23	`s0-s7`	saved registers	Callee saves/restores if used
24-25	`t8-t9`	temporaries / PIC patterns	`t9` often important in PIC calls
26-27	`k0-k1`	kernel/reserved	Not normal application scratch
28	`gp`	global pointer	ABI-specific global-data access
29	`sp`	stack pointer	Stack frame base for current call
30	`fp` / `s8`	frame pointer / saved	ABI and compiler dependent
31	`ra`	return address	Written by link operations such as `jal`

The **instruction set** defines what `zero` does; the **ABI** defines which registers a function should preserve and how arguments and results are conventionally passed. The CPU can execute a deliberately non-ABI-conforming function; it simply may not cooperate with compiled code that expects an ABI.

2.2 `zero`, `s0`, `v0`, and `ra` in practice

`zero` is architecturally always zero. For example, `addu t0, t1, zero` copies `t1` to `t0` modulo 32 bits. `s0` is often used to keep a pointer across function calls because an O32 callee that uses it is expected to

preserve the caller's original value. `v0` is a conventional return register but can hold intermediate values. `ra` is the register normally containing a call's return address after a linking jump. None of these purposes by itself reveals the higher-level name of a value.

Special execution state includes **PC** and, on classic implementations using them, **HI** and **LO** for multiplication/division results. CP0 exposes privileged status and exception state; it is not part of the 32 ordinary GPRs. ISA revision and target-specific extensions govern additional state.

2.3 Why operand order cannot be memorized as one universal rule

For arithmetic `addu t0, t1, t2`, the operands are destination, source1, source2; semantically `t0 = t1 + t2` with modulo arithmetic. For `lw v0, 4(s0)`, the **register** is the destination of a value read **from memory**. For `sw v0, 4(s0)`, the **register** is the source of a value written **to memory**. Both load and store are written `register, offset(base)`, so destination-first is not a universal syntactic rule.

```
addu t0, t1, t2    t0 <- t1 + t2
lw   v0, 4(s0)     v0 <- memory_word[s0+4]
sw   v0, 4(s0)     memory_word[s0+4] <- v0
```

A useful method is to ask three questions in order: (1) what operation is named? (2) which operands provide inputs? (3) what register, memory, or PC state is modified? For `sw`, neither `s0` nor `v0` is changed by the instruction; the memory destination is changed.

2.4 Register traces should be concrete

Consider the *synthetic* sequence:

```
addiu s0, zero, 0x1000   # s0 = 0x1000
addiu t0, zero, 5        # t0 = 5
addu  t1, s0, t0         # t1 = 0x1005
move  v0, t1             # pseudo: copy t1 into v0
```

After these instructions: `s0 = 0x1000`, `t0 = 5`, `t1 = 0x1005`, and `v0 = 0x1005`. `move` is typically a pseudoinstruction, represented by a real register operation; the disassembler's exact form may vary. The **numerical addition** does not tell us whether `t1` is a pointer, a counter, or a mistaken computation. Interpretation requires context.

An important hazard: the 16-bit immediate in `addiu` is **sign extended**, so `addiu t0, zero, 0xFFFF` gives `0xFFFFFFFF` in MIPS32, not 65,535. An assembly example can look obvious while hiding an immediate-width rule; always inspect the ISA semantics.

2.5 Calling-convention thinking begins here

Suppose a function receives a pointer in `a0`, copies it to `s0`, then calls another function. The likely intent is to keep a stable pointer after the call, because `a0` may be overwritten by the callee or reused for new arguments. This is an **ABI-based inference**, not a proof about what the pointer refers to. Confirm by tracing the value's definitions and dereferences.

```
function input a0 --copy--> s0 --(survives helper call)--> lw at +0x204
```

Worked example: fix the reader/writer misconception

Initial: `s0=0x3000`, `v0=0x55`, memory word `[0x3008]=0xAA`. Executing `sw v0, 8(s0)` writes `0x55` to memory `[0x3008]`; both registers stay unchanged. Starting again from the **original** state, executing `lw v0,`

`8(s0)` loads `0xAA` into `v0`; memory stays `0xAA`. These are *separate experiments*, not a sequential two-instruction program.

Stop and predict

1. What does the CPU do if an instruction writes to register `zero`?
2. Given `t1=0x20` and `t2=0x10`, predict `addu t0,t1,t2` and list which register changes.
3. Describe in words the direction of data movement in `lw t0,12(s1)` and `sw t0,12(s1)`.
4. Why does seeing `lw v0,0x204(s0)` **not** establish that `s0` points to `CCISM_SCP`?
5. What is the difference between a register's architecture-defined behavior and its calling-convention role?

Pass standard: state source, destination, and post-instruction state independently for a new code example.

Lesson 3. Loads, Stores, and Memory Access

Scheduled time: 35 minutes. Mastery objective: evaluate every common load/store, including byte widths and signedness, then determine what evidence identifies its backing memory.

3.1 One addressing form, opposite data directions

For a conventional MIPS load/store `OP rt, imm(rs)`, calculate an **effective address** by adding the sign-extended 16-bit immediate to the contents of base register `rs`. The immediate is measured in **bytes**, regardless of data width. `lw v0,4(s0)` advances four bytes, not four words.

`lw`: reads a 32-bit word into `rt`; `sw`: writes the low 32 bits of `rt` as a word to memory. For a MIPS32 teaching example, both work with four bytes. Loads overwrite the **destination register**. Stores overwrite the **destination memory**. Stores do not automatically change either source register.

```
load word:           store word:
Memory ---> v0       v0 ---> Memory
   ^                           ^
address=s0+off       address=s0+off
```

In a real system, address validity, alignment, permissions, memory type, and exception behavior matter. A clean mathematical effective address is not proof that the instruction can successfully access it.

3.2 Width and signedness table

Instruction	Memory bytes	What changes	Value handling
`lb rt,off(rs)`	1	`rt`	8-bit signed, sign-extend
`lbu rt,off(rs)`	1	`rt`	8-bit unsigned, zero-extend
`lh rt,off(rs)`	2	`rt`	16-bit signed, sign-extend
`lhu rt,off(rs)`	2	`rt`	16-bit unsigned, zero-extend
`lw rt,off(rs)`	4	`rt`	Full 32-bit word on MIPS32
`sb rt,off(rs)`	1	memory	Write low 8 register bits
`sh rt,off(rs)`	2	memory	Write low 16 register bits
`sw rt,off(rs)`	4	memory	Write low 32 register bits

For example, if a byte in memory is `0xE0`, `lb` produces `0xFFFFFFE0` in a 32-bit destination (signed -32); `lbu` produces `0x000000E0` (unsigned 224). The memory byte was **identical** in both cases. These differences often reveal whether a C field was declared `signed char` versus `unsigned char`, but compilers can also extend values for other reasons.

3.3 Why `lbu v0,0xE(v0)` is legal and important

This instruction uses `v0` both as the base input and destination output. The CPU obtains the **old** `v0` to form the address, then loads a new unsigned byte into `v0`. If old `v0=0x81002000`, it reads byte at `0x8100200E`. If that byte is `0xC0`, new `v0=0x000000C0`. The original pointer value is gone from `v0` after the instruction, unless separately preserved.

This creates a common reverse-engineering trap: reading several lines with one unchanging mental value for `v0`. The same register name can contain a pointer on one line and a boolean-sized or byte-sized value on the next.

3.4 Loads from C structure fields

Consider a *synthetic* 32-bit structure using four-byte integers:

```
struct status {
    uint32_t id;       // +0x00
    uint32_t flags;    // +0x04
    uint32_t counter;  // +0x08
};
```

If `s0` points to a `struct status`, `lw v0,4(s0)` corresponds to reading `status->flags`. But finding `lw v0,4(s0)` by itself does **not** prove the type of the object pointed to by `s0`. Several unrelated structures can have fields at `+4`. The type interpretation is a hypothesis supported by provenance, types, call context, observed values, and neighboring accesses.

A structure containing `uint8_t` fields may include **padding**, which means `sizeof(struct)` may exceed the sum of its field widths. Packing, target ABI rules, alignment, and explicit compiler attributes affect offsets. Do not infer C layout solely by counting declared members.

3.5 Following base pointers backward

In Ghidra, when you see `lw v0,0x204(s0)`, do not stop at the offset. Ask: where was `s0` last assigned? Was it copied from `a0`? Was `a0` loaded from global data? Was the pointer returned by an initializer? Does a caller pass the base of `CCISM_SCP`? If register flows cross a call, do you understand the ABI preservation rules? If you find a pointer's *virtual* address, can you establish how it maps to the region's modem-facing memory view?

```
0x204 access
   ^ depends on
s0 value at this instruction
   ^ copied from
argument/global/return value
   ^ initialized from
shared-memory descriptor or another object
   ^ validate
actual backing-memory mapping and execution path
```

Only after this chain is justified may you claim that the instruction reads your target field. Even then `lw` finds a **reader**, not necessarily the code that updates it. A store in this processor, a mapped alias, another CPU, or a DMA engine may be the writer.

3.6 A complete miniature trace

Initial synthetic state: `s0=0x4000FA00`, memory word at `0x4000FC04` is `0x00000040`, and the corresponding memory is readable under the current processor's mapping.

```
lw   v0, 0x204(s0)    # v0 <- M32[0x4000FC04] = 0x40
andi t0, v0, 0x40     # t0 <- 0x40 & 0x40 = 0x40
```

If the first value were `0x00000000`, the second result would be zero. The two instructions provide evidence of a **read and flag test**, *conditional on* the base address assumption. They do not by themselves prove the semantic label `SCREEN_OFF`, nor who writes the field.

3.7 Unaligned access, MMIO, and practical cautions

Word loads and stores generally require aligned addresses in classic MIPS. Some revisions provide `lwl/lwr/swl/swr` to assemble or scatter word fragments across aligned boundaries. They are subtle because endianness affects which bytes participate. Treat them with the ISA manual open, rather than relying on a simplified mnemonic. Byte-width accesses to device registers may be unsupported or have different side effects from RAM; MMIO can be read-to-clear or have ordering requirements. A debugger reading an address can perturb device state. Before experimenting on hardware, verify whether an address is ordinary RAM, shared RAM, device I/O, or an unmapped region.

Worked example: four instructions, four distinct effects

Initial, little endian: `s0=0x2000`, `t0=0x12345678`, bytes `[0x2004..0x2007] = FF 80 00 00`.

1. `lbu v0,4(s0)` sets `v0=0x000000FF`.
2. Starting from the same initial state, `lb v0,4(s0)` sets `v0=0xFFFFFFFF`.
3. Starting over, `lw v0,4(s0)` sets `v0=0x000080FF`.
4. Starting over, `sb t0,4(s0)` stores byte `0x78` at `0x2004`; the other three bytes stay as they were. `t0` stays `0x12345678`.

Stop and predict

1. `s0=0x3000`, `v0=0x12345678`, word memory `[0x3008]=0xAAAAAAAA`. Predict all state changes for `sw v0,8(s0)`.
2. Repeat for `lw v0,8(s0)`, starting from the **original** state.
3. A byte at `[0x7000]` is `0x80`: give results of `lb` and `lbu` into MIPS32 registers.
4. Explain why `lbu v0,0xE(v0)` must use old `v0` to compute the effective address.
5. Give two distinct ways a write to the target word could occur without there being an `sw ...0x204(s0)` in the inspected binary.

Pass standard: correctly predict register/memory changes, access width, signedness, and the limits of an apparent field match.

Lesson 4. Arithmetic, Logic, and Bit Manipulation

Scheduled time: 30 minutes. Mastery objective: reconstruct what arithmetic and bitwise instructions do, especially when a single flag controls a branch.

4.1 Arithmetic is arithmetic on fixed-width bit patterns

For MIPS32 `addu rd,rs,rt`, the destination receives the low 32 bits of the sum, modulo `2^32`. This instruction does not raise the signed arithmetic-overflow exception that `add` can raise. The suffix `u` in `addu` means **untrapped integer overflow behavior**, not that the inputs are intrinsically unsigned. The same register bits can be interpreted as signed or unsigned by other operations. `addiu` similarly does not trap on signed overflow, but its immediate is **sign extended**. `subu` computes the low 32 bits of a difference.

Instruction	Simplified classic MIPS32 meaning	Critical caution
`addu d,s,t`	`d = (s + t) mod 2^32`	no overflow trap
`add d,s,t`	signed addition	may raise overflow
`addiu t,s,k`	`t = s + sext16(k)` modulo `2^32`	`u` does **not** zero-extend immediate
`subu d,s,t`	`d = (s - t) mod 2^32`	fixed-width wraparound
`lui t,k`	`t = k << 16` on MIPS32	commonly starts 32-bit constant construction

A common compiler pattern for address construction is:

```
lui   t0, 0x4000       # t0 = 0x40000000
addiu t0, t0, 0xFA00  # BE CAREFUL: immediate is signed!
```

This sequence **does not** give `0x4000FA00`: `0xFA00` sign-extends to a negative immediate (`-0x600`), so the result is `0x3FFFFA00`. For the intended constant one simple form is:

```
lui t0, 0x4000         # 0x40000000
ori t0, t0, 0xFA00    # 0x4000FA00; ORI zero-extends immediate
```

When examining a compiler's `lui`/`addiu` pair, account for relocation-adjusted high halves. Assemblers often use a corrected high half when a low-half addend has bit 15 set. This is not a cosmetic detail: it changes actual addresses.

4.2 Bitwise logic: AND, OR, XOR, and NOT

AND preserves a bit only when both inputs have 1 in that position. **OR** sets a bit when either input has 1. **XOR** sets a bit when inputs differ. **NOR** inverts OR, subject to the register width. These operations are applied **bit by bit**, not to whole values as a single boolean. With a mask `0x40`:

```
value = 0xC1 = 1100 0001
mask  = 0x40 = 0100 0000
AND          ---------
result       0100 0000 = 0x40
```

`andi v0,v0,0x40` keeps only bit 6 of the old `v0`; every other bit becomes zero. If the result is nonzero, bit 6 was set. The value `0x40` is a mask, not evidence of an understood screen-off variable name.

By contrast, `ori t0,t0,0x40` sets bit 6 regardless of its previous value; `xori t0,t0,0x40` toggles that bit.

To clear a bit: `x & ~mask`, e.g., clearing bit 6 of a 32-bit value with `mask=0x40`. In assembly, an inverted mask may be constructed in another register or folded into suitable instructions. The bitwise complement is width-dependent: `~0x00000040` on 32-bit integers is `0xFFFFFFBF`.

4.3 Shifts and their meaning

A left shift `sll d,t,shamt` moves bits left, discards bits shifted off the top, and fills low bits with zero. A logical right shift `srl` fills high bits with zero. An arithmetic right shift `sra` replicates the sign bit at the high end, preserving the sign of a two's-complement value in the expected way. `sll t0,t1,2` commonly multiplies by four modulo 32 bits. A compiler can use that to index a table of four-byte elements.

```
8-bit illustration:  11110000 (0xF0)
logical >> 2:        00111100 (0x3C)
arithmetic >> 2:     11111100 (0xFC)
```

Rotate instructions, bitfield extract/insert instructions, and variable shift forms exist in applicable MIPS revisions. Never assume a mnemonic is present on every MIPS core; disassembler language selection matters.

4.4 Comparisons and predicate formation

`slt d,s,t` sets `d=1` when signed `s<t`, otherwise `d=0`. `sltu` makes the comparison **unsigned**. `slti`/`sltiu` compare against a sign-extended immediate according to ISA rules. A branch can then examine the predicate register. One common pattern is:

```
sltu t0, a0, a1       # t0=1 if unsigned a0<a1
bne  t0, zero, less   # branch if comparison true
nop                   # classic delay slot
```

Do not equate the `u` in `sltiu` with zero-extending its immediate; compare the actual documented operand semantics. Comparisons often encode signedness choices originating in C source code.

4.5 Reconstruct the real-world flag test

Consider the following *illustrative*, classic MIPS32 sequence:

```
lw   v0, 0x204(s0)      # 32-bit word from base+0x204
andi v0, v0, 0x40       # keep bit 6
beq  v0, zero, clear    # branch if bit 6 was clear
nop                     # delay slot executes regardless
```

If the memory word is `0x40`, after the load `v0=0x40`, after `andi v0=0x40`, and `beq` is **not taken**. If the word is `0`, after AND `v0=0`, and the branch **is taken**. The instruction sequence allows you to infer that execution distinguishes the bit's state. It does not establish what the bit means; it also may be only one of many tests of that field.

Worked example: bit operations without semantic guesses

Initial: `t1=0x000000A5` (`1010 0101`), `t2=0x40` (`0100 0000`). `and t0,t1,t2` produces zero because bit 6 of `0xA5` is clear. `or t0,t1,t2` produces `0xE5`; `xor t0,t1,t2` also produces `0xE5` here because bit 6 was

initially clear. If bit 6 were already set, OR would leave it set whereas XOR would clear it.

Stop and predict

1. Predict `0xC1 & 0x40`, `0xC1 | 0x40`, and `0xC1 ^ 0x40`.
2. Why is `addiu t0,zero,0xFFFF` not the same as putting unsigned 65,535 into `t0`?
3. With `v0=0x80`, what value results from `andi v0,v0,0x40`?
4. Given unsigned `a0=0xFFFFFFFF` and `a1=1`, what result does `sltu t0,a0,a1` produce? What about signed `slt`?
5. Explain why `andi` followed by a branch is evidence of a bit test but not proof of the bit's semantic name.

Pass standard: show bit-by-bit calculations and predict control decisions without assigning invented meaning to a bit.

Lesson 5. Branches, Jumps, and Execution Flow

Scheduled time: 35 minutes. Mastery objective: trace a real instruction stream correctly, including classic delay slots and the difference between branching and calling.

5.1 Branches make control flow conditional

`beq rs,rt,label` branches if the two register values are equal; `bne` branches if unequal. `beqz rs,label` and `bnez` are usual pseudoinstruction spellings of branch-on-equal/not-equal with `zero`. `beq` compares **bit-pattern equality**, not signed versus unsigned magnitude. A source-like `if (x==0)` often becomes `beq x,zero,branch` or its inverse. A `while` loop can appear as a conditional branch, a body, and a backward jump.

Classic I-type branch immediates are **signed 16-bit word displacements**. For classic 32-bit branch instructions, the branch target is computed as `PC + 4 + (sign_extend(imm16) << 2)`. The immediate does not contain an absolute 32-bit byte address. For example, from branch instruction address `0x1000`, displacement `+3` targets `0x1004 + 12 = 0x1010`. When applicable, the instruction at `0x1004` is the **delay slot**, which executes before transfer to `0x1010`.

5.2 Direct jumps, indirect jumps, and links

`j target` jumps unconditionally using a compact target field; the exact classic MIPS32 encoding constructs the destination from a shifted 26-bit field and high PC-region bits. This imposes region limits on what a single `j` can reach. `jr rs` jumps to the address held in a register and is commonly used for returns. `jal target` performs a jump *and* saves a return address in `ra`. `jalr` performs a register-indirect linking jump, often for function pointers or position-independent code.

For classic delayed-call instructions, `jal` normally writes the address **after its delay slot** (`PC + 8`) into `ra`. If the call is at `0x80001000`, the next instruction at `0x80001004` is its delay slot, and the return point is `0x80001008`. These rules depend on the exact architecture revision and instruction mode. **Release 6 and compact branches are not equivalent to classic delay-slot instructions.**

```
0x80001000: jal  helper
0x80001004: nop            <- executes before helper starts
0x80001008: addiu v0,v0,1  <- return address in classic MIPS

helper:
    ...
    jr   ra
    nop
```

The pair `jr ra; nop` forms a conventional return because `ra` contains a return address and `jr` uses it as the jump destination. The `nop` occupies the return's delay slot on classic MIPS, preventing an unintended instruction from executing in that slot. A compiler may instead place a useful safe instruction there, such as stack-pointer restoration, if allowed by dependencies and scheduling.

5.3 Delayed branches: what actually executes

For classic MIPS with delay slots, the instruction **immediately following** a delayed branch executes before the branch target receives control, whether the usual `beq` is taken or not. This frequently surprises readers trained on other architectures. A branch-likely instruction can *annul* the delay slot when not taken; its behavior differs and availability depends on revision. A branch

or jump **inside another delay slot** can have restricted or unpredictable behavior; do not casually patch delay slots with new control transfers.

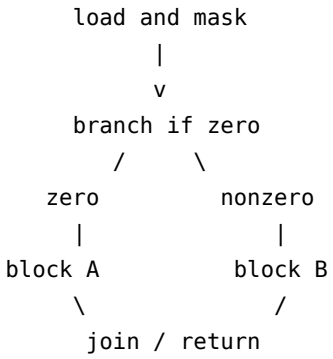
Illustrative example:

```
0x1000: beq  t0, zero, target
0x1004: addiu t1, t1, 1       # classic delay slot
0x1008: addiu t2, t2, 7       # fallthrough only if not taken
...
target:
0x1020: addiu t3, t3, 1
```

If `t0=0` and initial `t1=9`, the CPU executes the branch, then increments `t1` to 10 at `0x1004`, then goes to `0x1020`. Instruction `0x1008` is skipped. If `t0` is nonzero, `0x1004` still runs, and execution continues at `0x1008`.

5.4 Control-flow graph reasoning

A **basic block** is a straight-line sequence with a single entry and (typically) a final control-transfer decision. Compilers and Ghidra use blocks and directed edges to represent possible flow. When analyzing firmware, draw branches as **conditional alternatives** and calls as transfers that normally return. An indirect jump may have targets that cannot be determined statically; keep unknown destinations visible instead of inventing one.



Branch analysis is essential for finding whether a certain memory read is reached under specific screen states. An instruction's presence in a binary does not mean the CPU executes it during the experiment.

5.5 Calls and return values

A standard O32 function receives its initial integer/pointer arguments mostly in `a0` through `a3`; it usually returns a scalar in `v0`. `jal` modifies `ra` as part of its control-transfer semantics, so a function that makes its own calls normally saves the incoming return address before overwriting it. This is why prologues often include `sw ra,...(sp)`. `ra` is not a separate call stack; nested calls require explicit preservation.

5.6 Debugger consequences of delay slots

Consider a debugger that reports a trap while the processor is executing a branch's delay slot. In relevant classic MIPS exception models, exception state can indicate that the fault occurred **in a delay slot** and EPC can refer to the **branch instruction**, not simply the trapping slot's address. Incorrectly assuming that EPC always names the faulting instruction makes resume and single-

step logic wrong. The exception flow must be interpreted with the privileged architecture manual for the actual ISA revision.

Worked example: explain three successors

Initial: classic MIPS32, `t0=0`, `t1=4`, `t2=5`.

```
0x2000: beq   t0, zero, 0x2010
0x2004: addiu t1, t1, 1
0x2008: addiu t2, t2, 1
0x200C: nop
0x2010: addu  v0, t1, t2
```

The branch is taken, but its delay slot at `0x2004` executes and makes `t1=5`. `0x2008` and `0x200C` are skipped. At `0x2010`, `v0=5+5=10`. The statement "the branch skips the immediately following instruction" is therefore wrong on this classic target.

Stop and predict

1. A delayed `jal` at `0x3000` has a delay slot at `0x3004`. What return address does `ra` receive in classic MIPS32?
2. If `beq t0,zero,L` is taken, will a normal instruction in its delay slot execute?
3. Why might `jr ra` be followed by `nop`?
4. Given a classic branch at `0x1000` with signed displacement `+3`, compute the destination address.
5. Why is a debugger's simplistic rule "advance PC by four after every breakpoint" dangerous?

Pass standard: trace both the taken and untaken paths and explicitly show delayed execution.

Lesson 6. Functions, Stack, and Calling Conventions

Scheduled time: 35 minutes. Mastery objective: reconstruct a function's arguments, saved state, local stack frame, and return behavior from assembly.

6.1 The stack is a memory discipline, not a magic CPU object

On typical MIPS ABIs, the stack grows toward **lower addresses** as a function reserves space: subtract a positive size from `sp` to allocate a frame. The ABI defines alignment, saved-register placement conventions, outgoing argument areas, and caller/callee obligations. A particular instruction sequence is not a "stack operation" by hardware magic; it simply computes addresses relative to `sp` and loads/stores memory there.

```
higher addresses
+------------------+
| caller's frame   |
+------------------+ <- old sp
| saved ra         |
| saved s0         |
| local storage    |
| outgoing args    |
+------------------+ <- new sp
lower addresses
```

The actual order and sizes differ by function, compiler, and ABI. Always derive offsets from the real prologue, rather than imposing this diagram as an exact template.

6.2 O32 as a concrete convention

For many classic MIPS32 binaries using O32, the first four integer/pointer argument words use `a0-a3`, with further arguments on the stack. `v0-v1` carry usual integer results. The `t` registers are caller-saved; the `s` registers are callee-saved. `ra` must be preserved by a callee that overwrites it and still needs to return to its own caller. O32 often requires 8-byte-aligned stack frames and a caller-provided four-word (`16-byte`) home area for argument registers; consult the actual ABI rules in all edge cases, including hard-float conventions, 64-bit arguments, structures, and variadic functions [M4]. **N32/N64 differ** and may use additional argument registers.

The callee-saved rule is a contract between functions. If a function writes `s0`, it must restore the caller's `s0` before returning, even if it uses `s0` internally as a structure pointer. "Saved" does not mean the register can never change; it means the value must be preserved **across the function call**.

6.3 A realistic function prologue and epilogue

A *synthetic* classic O32-like function reads a status field and returns whether bit 6 is set. `a0` is assumed to point to a valid object:

```
check_flag:
    addiu sp, sp, -24     # reserve 24 bytes (8-byte aligned)
    sw    ra, 20(sp)      # save return address
    sw    s0, 16(sp)      # preserve caller's s0
    move  s0, a0          # retain incoming pointer
```

```
    lw    v0, 0x204(s0)   # load status word
    andi  v0, v0, 0x40    # isolate bit 6
    sltu  v0, zero, v0    # v0 = 1 if result nonzero
    lw    s0, 16(sp)      # restore caller's s0
    lw    ra, 20(sp)      # restore return address
    jr    ra              # return to caller
    addiu sp, sp, 24      # classic return delay slot
```

This example is written for a classic delay-slot ISA and an assumed access width, stack mapping, and ABI. Its final stack adjustment deliberately executes in the `jr` delay slot. On another ISA revision, blindly copying this sequence would be unsafe. The function makes no calls itself, so saving `ra` is redundant for this exact leaf example but demonstrates the preservation pattern; optimizing compilers may omit it and even omit the frame.

Read it mechanically: incoming `a0` is copied to `s0`. `lw` uses `s0` as base; `andi` tests bit 6; `sltu` converts any nonzero mask result to integer 1; restore `s0` and `ra`; return. Before entry `sp` might be `0x80002000`; after allocation it is `0x80001FE8`; `ra` is stored at `0x80001FFC`; `s0` at `0x80001FF8`. The `jr` delay slot restores `sp` to `0x80002000` before the caller resumes.

6.4 `ra` is not a nesting mechanism by itself

A second `jal` overwrites `ra`; therefore a function that calls a helper and later returns to its caller typically saves the first return address. If it does not, `jr ra` may jump back to the wrong point, creating a loop or crash. Optimizations can avoid a save under special conditions, such as tail calls or when another saved location carries the return address. Interpret actual control flow, not just template prologues.

6.5 Stack spills and local variables

Compilers have a finite number of registers. Values may be **spilled** to stack slots with `sw`, then reloaded with `lw`. These memory operations look like structure accesses, but their base is `sp` or `fp` and they are often local temporaries. Recognizing stack-relative accesses prevents you from confusing a function's own frame with global shared memory. Frame pointers may be omitted, and a compiler can refer to all locals relative to a changing `sp` while preserving correctness.

6.6 Derive argument meaning from callers

Seeing `move s0,a0` means only that the incoming `a0` value is preserved in `s0`. To identify what object it points to, find the call site, track the value placed in `a0` before `jal`, and resolve any global lookup or initializer that produced it. Repeat across multiple callers. Applying this to `lw v0,0x204(s0)` may establish whether it references `CCISM_SCP` or merely a coincidental offset.

6.7 ABI complications that matter to debuggers

A debugger collecting registers at arbitrary instructions cannot assume argument values still reside in `a0-a3`; the compiler may have reused them. It cannot assume `ra` always equals the correct previous frame's return address; optimized and asynchronous contexts complicate unwinding. Stack corruption, signal/interrupt frames, position-independent code, and mixed ABIs require careful reasoning. A calling convention helps predict *valid* code, but debugger context capture must preserve the **actual machine state**, not an idealized function signature.

Worked example: trace frame arithmetic

Initial: `sp=0x90001000`, `ra=0x80001234`, `s0=0xAAAA5555`. `addiu sp,sp,-24` makes `sp=0x90000FE8`. `sw ra,20(sp)` writes `0x80001234` at `0x90000FFC`. `sw s0,16(sp)` writes `0xAAAA5555` at `0x90000FF8`. Later restoring these slots and adding 24 to `sp` recovers the caller-visible state.

Stop and predict

1. With `sp=0x80001000`, after `addiu sp,sp,-32`, what is the new `sp`?
2. At new `sp`, what exact address is `sw ra,28(sp)` writing?
3. What must a well-behaved O32 callee do if it modifies `s2`?
4. Why can an `lw` using `sp` as its base be a local variable load rather than a shared-memory read?
5. Why is examining the caller necessary to determine what type of pointer arrived in `a0`?

Pass standard: draw a stack frame from an actual prologue, and recover register values at return.

Lesson 7. Machine Code, Ghidra, and Reverse Engineering

Scheduled time: 30 minutes. Mastery objective: decode a basic MIPS instruction from raw bytes and choose Ghidra operations based on a clear evidentiary question.

7.1 Assembly is a readable representation of bits

The processor executes **machine instructions**, not textual mnemonics. The mnemonic `lw` is a human-readable name for a particular encoded operation. A classic MIPS32 instruction is normally 32 bits. R, I, and J formats partition these bits into fields. An instruction's high six bits often contain a major opcode, followed by register selections, an immediate, or a function selector.

```
Classic R type: [ opcode:6 | rs:5 | rt:5 | rd:5 | shamt:5 | funct:6 ]
Classic I type: [ opcode:6 | rs:5 | rt:5 | immediate:16       ]
Classic J type: [ opcode:6 | target:26                        ]
```

Pseudoinstructions such as `move`, `li`, `la`, `bnez`, and `nop` may assemble to one or more real encodings according to context. A disassembler might display a pseudoinstruction even though the raw bits encode a more primitive operation. Conversely, a mistaken ISA-mode choice can cause plausible-looking but **false disassembly**.

7.2 Decode `lw v0,0x204(s0)` manually

For a classic MIPS32 `lw` I-type encoding:

- `opcode = 0x23` (decimal 35; binary `100011`).
- `rs = s0 = register 16 = binary 10000`.
- `rt = v0 = register 2 = binary 00010`.
- `imm16 = 0x0204`.

Put fields together:

```
 opcode   rs     rt         immediate
 100011 | 10000 | 00010 | 0000001000000100
       full instruction word: 0x8E020204
```

In little-endian memory the **instruction's four bytes** are `04 02 02 8E`. These are machine-code **instruction bytes**, not the four data bytes that the instruction loads when it executes. A good reverse engineer differentiates instruction bytes, disassembled operations, effective memory addresses, and run-time memory contents. For `sw v0,0x204(s0)` the encoding instead uses opcode `0x2B`; the word is `0xAE020204`, or bytes `04 02 02 AE` in little endian.

This is why a literal search for `0x204` is not a guaranteed way to find all accesses: constants may be reconstructed, copied through registers, folded into other offsets, or omitted in optimized code; instruction immediates may also occur coincidentally in unrelated functions.

7.3 Decode a branch destination

For a classic instruction `beq t0,t1,target`, the opcode is `0x04`; `rs=t0=8`, `rt=t1=9`; the final field is a signed 16-bit **word displacement**. To determine the taken destination, start from the address of the branch instruction, add four, then add `sext16(imm) << 2`. The instruction at `PC+4` is a delay slot in the classic architecture. An ISA revision using compact branches requires different treatment.

A `j` or `jal` contains a 26-bit field that is shifted left by two, with upper target-region bits constructed from `PC+4` in classic MIPS32. It is not a full, independent 32-bit absolute address. For jumps outside

a reachable region, compilers or assemblers may synthesize a register-indirect sequence.

7.4 Start with the *question*, then choose the tool

Ghidra is a static-analysis environment: its **Listing** shows decoded instructions and data, its **Decompiler** produces a reconstructed high-level approximation, its **Symbol Tree** organizes names, and **XREFs** connect references. Static analysis does not automatically show which instruction ran during a specific screen transition. The following decision table is more useful than randomly clicking symbols.

Investigation question	Useful Ghidra feature	Important limitation
Where is this exact byte pattern?	Search -> Memory	Can also match data or unrelated code
Where is this numeric immediate?	Search -> For Scalars	May miss computed constants or alternate forms
Who references a known string?	Strings and references	Strings may be unused or shared
Who calls this function?	XREFs / Call Tree	Indirect calls can be missed
What instructions execute in this function?	Listing / Function Graph	Depends on correct disassembly
What C-like operations might this be?	Decompiler	Types and control flow may be guessed
What bytes and permissions are mapped?	Memory Map	Image layout may differ from run-time mapping
What does `s0` derive from?	Listing + backward slice, callers	Requires human or p-code data-flow reasoning
How is a field laid out?	Data Type Manager, decompiler	Depends on correct layout and ABI assumptions

A **symbol** names an address or program object. Function symbols may name entry points; data symbols may name variables or tables. Symbols recovered from firmware, debug information, or exports may be meaningful, whereas autogenerated names like `FUN_...` or `DAT_...` usually indicate uncertain identity. A name is evidence of intended purpose only when its provenance is credible; it is not a guarantee that a function is safe or relevant.

7.5 What an XREF does and does not mean

An XREF is a reference from some code/data location to another location or symbol. A function cross-reference may indicate a caller; a data reference may point to an address or string. XREFs are useful to navigate *known* relationships, but indirect loads, computed addresses, register-indirect calls, relocation effects, and data-driven dispatch may not produce simple explicit XREFs. Absence of an XREF is not proof of absence of use [T1].

Ghidra's intermediate **p-code** models data transfers such as LOAD, STORE, arithmetic, and control flow, and underlies much analysis [T2]. It can help reason about where values came from, but interpretation still depends on a correct processor language, flow recovery, and data-type assumptions.

7.6 A reliable backward trace

Suppose `lw v0,0x204(s0)` is seen in a function. We wish to know whether it accesses `CCISM_SCP + 0x204`.

1. **Record the exact instruction**, address, and function name. Confirm it is code in the right architecture mode.
2. **Locate the last definition of `s0`** that reaches the load. If it was `move s0,a0`, the question becomes where `a0` came from.
3. **Inspect the caller** and the instructions preparing `a0`, respecting delayed branches and calling convention rules.
4. **Follow global lookups and initialization**, including any table where offsets are computed at runtime rather than encoded as fixed values.
5. **Identify the processor's address view**. A modem-view address, AP physical address, and AP virtual mapping may refer to shared backing storage but need not have identical numbers.
6. **Look for independent confirmation** through types, neighboring fields, runtime accesses, known structures, or cross-processor source.
7. **Keep uncertainty explicit:** is it only a candidate, a proven reader, or a proven writer? Does a trace demonstrate execution at the relevant time?

The method generalizes to any register and any structure. The difficult work is often tracing *what base address means*, not the addition of `0x204`.

7.7 Common decompiler and symbol traps

The decompiler may label an argument `param_1`, a pointer `undefined4 *`, or a variable with a type inferred from usage. Its apparent `structure->field` could result from an incorrect type that *you* assigned. A function name containing `debug`, `status`, or `trace` may be relevant to your objective, but a name alone does not establish invocation timing or relation to the shared-memory field. Inspect call sites, inputs, outputs, side effects, and data provenance before investing heavily. A very long function may be a dispatcher, a runtime library routine, or initialization logic; ask what evidence makes it relevant.

Worked example: verify instruction bytes

Classic MIPS32 little-endian: raw bytes `04 02 02 8E` correspond to instruction word `0x8E020204`. `opcode=(word >> 26) & 0x3F = 0x23`, `rs=(word >> 21) & 31 = 16`, `rt=(word >> 16) & 31 = 2`, and `imm=word & 0xFFFF = 0x0204`. The decoded operation is `lw v0,0x204(s0)`. The execution-time address is still unknown until `s0` is traced.

Stop and predict

1. Why does a hex dump show `04 02 02 8E` for the 32-bit instruction word `0x8E020204` on a little-endian target?
2. Name the four fields of the I-type load encoding and their bit widths.
3. You see `move s0,a0` immediately before the interesting load. What do you search or inspect next, and why?
4. Why is a scalar search for `0x204` neither necessary nor sufficient to prove access to a field at `+0x204`?
5. What additional proof is needed before claiming the instruction *executed* when the screen changed?

Pass standard: decode one load by hand and produce a defensible two-step provenance investigation.

Lesson 8. Exceptions, Breakpoints, and Debugger Internals

Scheduled time: 35 minutes. Mastery objective: explain which machine state changes during a trap, which state must be saved, and how a correct debugger avoids retriggering or corrupting execution.

8.1 What a debugger must be able to do

A debugger is an **observer and controller of program execution**. The minimum architectural capabilities are memory inspection, register inspection, execution stop/resume, and a way to report an event. Code editing, breakpoint insertion, step controls, and a user interface are built on top of these primitives. A **host debugger** can communicate with a target-side **stub/agent**; the stub performs operations in the target's execution and memory environment. An external debug probe instead uses hardware access mechanisms. These are different architectures with different tradeoffs.

```
Host debugger <---- transport ----> Target debug stub
  symbols/UI                              |
  breakpoints                             +-- read/write memory
  stack view                              +-- read/write registers
                                          +-- stop/resume
                                          +-- exceptions/events
```

Reading memory is not necessarily safe in every region: an unmapped read may fault, and some MMIO addresses have side effects. Writing executable memory is insufficient by itself when instruction caches hold old code. Reading registers **after** an exception is only meaningful if their values were preserved before handler code overwrote them.

8.2 Exceptions and CP0 concepts

An **exception** diverts the CPU from ordinary flow to a handler. Examples include instruction-address errors, data-address errors, reserved instructions, system calls, traps, and breakpoints. **Interrupts** are normally asynchronous events; a software breakpoint is a synchronous exception triggered by executing an instruction such as `break`, if allowed on the target. In classic MIPS32, CP0 exposes privileged state related to **Status**, **Cause**, **EPC**, and often **BadVAddr**. These names identify roles; actual accessibility and behavior require the relevant privileged architecture manual [M3].

- **Status:** selected execution privilege, interrupt and exception control state.
- **Cause:** information about why exception processing began; includes exception code and, on classic delayed branches, branch-delay indication.
- **EPC:** exception program counter used for restart/return decisions.
- **BadVAddr:** associated invalid or faulting virtual address for relevant exceptions; not meaningful for every possible exception.
- **Exception vector:** entry location at which handler instructions begin, dependent on CPU and configuration.

A handler typically needs a safe early entry path to save state without accidentally clobbering user/firmware registers. Exception stack setup, nested exceptions, interrupt masking, and privileged register access are significant design issues, not mere implementation details.

8.3 Why EPC is not always simply “the PC that faulted”

On relevant classic MIPS delay-slot exception behavior, **when an exception occurs in a branch delay slot, the BD indication is set and EPC can contain the address of the branch**, not the delay-slot instruction. Resume logic must reconstruct the correct control-flow context from this state. Treating EPC as a universal `trapping_address` is a serious debugger bug.

Illustration:

```
0x80001000  beq t0,t1,target
0x80001004  break             # delay slot: special case
0x80001008  fallthrough
```

A `break` in the delay slot is not equivalent to `break` at an ordinary instruction boundary from the perspective of exception restart and branch flow. Other ISA revisions have different branching and exception contracts. A debugger must branch on the *actual architecture*, not on an abstract claim that all MIPS has identical delay slots.

8.4 Software breakpoint lifecycle

A software breakpoint is not just a trap word placed in memory. It is a **stateful protocol** between target, handler, and host. The following is conceptual for one classic 32-bit instruction location `X`:

```
1  read and save original instruction at X
2  write BREAK encoding at X
3  make executable instruction stream coherent
4  resume target; CPU reaches X and traps
5  handler saves exact execution context
6  report event and original instruction address
7  restore original instruction at X
8  execute original instruction once (or safely emulate)
9  reinstall breakpoint if still enabled
10 make code coherent and continue
```

Why each step matters: **saving** allows restoration; **patching** causes the stop; **cache maintenance** ensures the fetch side sees the patched instruction; **context capture** makes inspection meaningful; **restoration and execute-once** avoid trapping forever at the same PC; **reinstallation** preserves a persistent breakpoint. On multicore targets, another core can enter the same location while it is temporarily unpatched, so synchronization and per-core state matter.

Step 8 is often the hardest. It may require hardware single-step, a carefully placed temporary breakpoint, correct branch emulation, or controlled execution of a relocated instruction. **Merely incrementing the PC by four skips the replaced instruction**, changing program behavior. Replacing it with a `nop` has the same correctness problem if the original had effects.

8.5 Hardware breakpoints and watchpoints

Hardware instruction breakpoints can detect an instruction fetch or execution without rewriting that instruction, subject to implementation limits. **Data watchpoints** can monitor selected reads/writes, valuable for finding the writer of a memory word. Availability, number of comparator slots, alignment, access-type filters, debug mode, and address translation vary across cores. MIPS debug features, including EJTAG-related capabilities on supporting systems, must be **verified**, not presumed. Access to a running target’s debug facilities may require privileged firmware or physical interfaces unavailable in the project environment.

8.6 Reading CPU context accurately

An interruption can occur while a function has partially built a stack frame or is in a delay slot. The debugger needs a **snapshot at the correct point in execution**, typically including 32 GPRs, PC/restart state, HI/LO if relevant, privilege/exception state, and optionally FP/DSP state, depending on the target. A host GDB client expects a register layout matching the target description; wrong widths or ordering can produce convincing but incorrect disassembly and backtraces.

Saving `s0` after the handler has already used `s0` loses the original value. Saving context on a corrupted application stack may destroy evidence or cause nested faults. These concerns motivate low-level entry stubs, reserved scratch state, and carefully designed exception stacks.

8.7 Correct resume is an architectural action

Before returning to the interrupted program, the debugger may need to restore original instruction bytes, arrange a safe one-instruction execution, restore register values, resolve PC/EPC and delay-slot state, synchronize caches, and update debug-control status. The exact privileged return (`eret` in relevant modes) and state restoration sequence are architecture- and handler-dependent. A correct debugger does not simply "jump back to EPC" without understanding which exception state must be cleared and whether the branch instruction must be retried.

8.8 GDB remote protocol in context

The GDB Remote Serial Protocol (RSP) provides a host-to-stub exchange of packets framed like `$payload#checksum`. Common packet families include `g/G` (read/write registers), `m/M` (read/write memory), `c/s` (continue/step), `?` (stop reason), and `Z/z` for breakpoint/watchpoint operations where supported. The exact semantics, extensions, register map, checksums, and acknowledgments are specified in the GDB manual [T3]. Supporting a packet name does **not** magically provide the underlying hardware or software mechanism; the stub must implement it correctly.

Worked example: avoiding an infinite breakpoint loop

Original instruction at `X`: `addiu t0,t0,1`; debugger replaces it with `break`. When the CPU reaches `X`, exception handling reports a breakpoint. If the handler simply resumes at `X` **without restoring** the original instruction, the CPU hits `break` again. If the handler instead resumes at `X+4` without executing the original, `t0` never increments. A correct implementation must execute the original instruction's effect exactly once while preserving the debugger's ability to stop there again later.

Stop and predict

1. Why must a software debugger save the original instruction bytes before patching in `break`?
2. What goes wrong if the debugger resumes at `X+4` after replacing `addiu t0,t0,1` at `X`?
3. Why is `EPC == address_of_faulting_instruction` an unsafe universal assumption on classic delayed-branch MIPS?
4. Which debugger capability would you prefer for identifying the **writer** of a shared word: instruction breakpoint on an unrelated `lw`, or a supported write watchpoint on the field? Explain.
5. Why might a successful write of breakpoint bytes to memory still fail to produce a trap?

Pass standard: narrate the entire breakpoint sequence, including restoration, execution-once, cache issues, and context preservation.

Lesson 9. Hooks, Patching, and the MediaTek Application

Scheduled time: 25 minutes. Mastery objective: explain how execution is redirected and returned, identify patch hazards, and construct a falsifiable evidence plan for `CCISM_SCP + 0x204`.

9.1 Patching changes executable behavior, not just bytes

A **patch** replaces instruction bytes at a specific code location. It must respect instruction boundaries, ISA mode, branch reachability, mapping permissions, atomicity, and instruction-cache visibility. Writing four bytes is **not** safe if the target uses a 16-bit/mixed-width instruction mode or if the four-byte region overlaps a branch delay slot whose behavior must be preserved. The method of performing the write and the correct cache maintenance sequence are target-specific.

A **detour** changes a function or code path so control reaches new code. An **inline hook** is a common detour placed by modifying instructions in the original function. A **trampoline** is a small code sequence that often preserves or reproduces displaced instructions, invokes instrumentation, and rejoins the original program after the patched region. The names describe roles, not a guarantee of safe implementation.

```
Original:  A --> B --> C --> D

Patched:   A --> jump_to_hook
                      |
                      v
                 hook code
                      |
                      v
           displaced B/C as needed
                      |
                      v
                      D
```

9.2 Displaced instruction relocation is the hard part

If a patch replaces an instruction that computes a PC-relative branch, a jump, or another location-sensitive operation, copying its bytes into a trampoline may change its target or semantics. On classic MIPS, displaced instructions might include branch-delay-slot behavior, a `jal` that writes a return address, or a pair of instructions that jointly construct a pointer. Trampolines therefore require **instruction-aware relocation**, not just `memcpy`. The same holds for restoring execution state: if hook code clobbers `t0` or `ra`, the original program may fail much later for reasons that appear unrelated.

Another issue is **control-transfer range**: a single `j` instruction cannot always reach an arbitrary 32-bit target on classic MIPS. Longer sequences may construct an address and use `jr`, consuming scratch registers and additional bytes. If those bytes overwrite useful instructions, relocation complexity grows. Prefer a formally reasoned design and test it on isolated code before applying it to sensitive firmware.

9.3 Instruction cache and multiple processors

A code patch is not necessarily visible immediately to an instruction fetch. Data writes and instruction-cache contents can diverge; an appropriate architecture- and platform-specific

sequence may require data-cache writeback, instruction-cache invalidation, barriers, and synchronization with other cores. Never assume that a data read confirming patched bytes proves that the next instruction fetch will execute them. Shared execution, interrupt activity, and another core's instruction cache can create difficult races.

9.4 The MediaTek shared-memory case: establish what is known

The previously supplied project notes describe the following observations and reconstructed layout:

```
Major Region 2 (modem view): 0x40000000
Candidate CCISM_SCP start:   +0xFA00
Observed changing word:     +0xFC04
Relative field displacement: 0xFC04 - 0xFA00 = 0x204
Candidate modem-view address: 0x4000FC04
```

The observations were **screen OFF -> `0x00000040`**, **screen ON -> `0x00000000`**, repeated with no intermediate values seen and no neighboring word changed in a small comparison. This supports a reproducible correlation under the tested conditions. It does not identify the flag's official meaning, rule out unobserved rapid transitions, or identify the processor or instruction that writes it.

A descriptor table and initialization loop allegedly build region offsets cumulatively as `current.offset = previous.offset + previous.size`. That explains why source initializers with `offset=0` can still correspond to runtime-assigned subregion offsets. To verify the ownership claim, re-create the table order, account for sizes/alignment/gaps, and confirm that the computed `CCISM_SCP` interval actually **contains** offset `0xFC04`.

9.5 Two coordinate systems, one backing memory?

A physical buffer can be visible at different numeric addresses to different processors. **Modem-view** `0x4000FC04`, an **AP physical** address, and a **kernel virtual mapping** may refer to the same backing storage if the platform's remapping and page tables establish that equivalence. Do not add `0x20000000` to every address by assumption: start with actual mapping metadata. Also distinguish a *region's* base from a *subregion's* base and from a pointer into the middle of a structure.

This is an important general lesson: an `lw` or `sw` immediate of `0x204` is just a 16-bit displacement. It says nothing about physical memory ownership until the **base register**, active address space, and mapping are established. A store by the application processor to its alias might update the word observed by modem firmware without any MIPS `sw` accessing a `0x4000...` address.

9.6 A defensible investigative decision tree

Start with one exact question: **Which code or device updates the observed word during a screen-state change?**

1. **Reproduce** the value transition and record the processor view, timing, access width, and observation tool.
2. **Validate the mapping** from the observed address to the backing region and calculated `CCISM_SCP` subregion.
3. **Identify readers** in modem disassembly only after proving the base register's provenance. `lw` may show consumption of the flag.
4. **Identify candidate writers** using stores, call paths, AP-side sources, or a supported data-write watchpoint. Remember that writes may use aliases, other widths, structures, memcpy-

like helpers, or DMA.
5. **Trace execution**, if permitted and feasible, to establish that a writer executes during the transition.
6. **Determine semantics** separately: find constants, field definitions, protocol documentation, and experiments that distinguish `SCREEN_OFF` from `LOW_POWER_REQUEST`, `SCP_STATE`, or other plausible meanings.
7. **Record evidence by strength**: observed, inferred, hypothesized, falsified. Never upgrade a hypothesis because an offset happens to match.

9.7 Build debugger capabilities in safe layers

A reasonable conceptual sequence is: (1) visibility into target memory and registers; (2) controlled proof of executable-code modification; (3) controlled exception observation; (4) breakpoint trap and saved context; (5) correct resume/step mechanism; (6) stable host-stub communication; (7) multiple breakpoints, watchpoints, and instrumentation. Each stage has a measurable invariant. For example, after pausing at `X`, a register value must match the **pre-handler** state. After stepping the original instruction, its effect must occur exactly once. When no breakpoint is set, the original code path should remain unchanged.

A debugger that can patch and trap but cannot resume correctly is **not yet a general-purpose debugger**. A hook that redirects execution but loses one saved register is **not correct instrumentation**. Correctness is demonstrated with tests, not by a successful one-time printout.

9.8 Final guided analysis

Consider this *synthetic* code, **not confirmed firmware**:

```
handle_state:
    lw    v0, 0x204(s0)
    andi  v0, v0, 0x40
    beq   v0, zero, screen_active_path
    nop
    jal   notify_helper
    nop
screen_active_path:
    jr    ra
    nop
```

What can you conclude? It *reads* a word at `R[s0]+0x204`, isolates bit 6, and chooses whether to call `notify_helper`. What **cannot** you conclude? That `s0` is `CCISM_SCP`; that the word formally means `SCREEN_OFF`; that this function executes during an actual transition; or that this function writes the field. An important further concern: this sketch overwrites `ra` with `jal` and later returns via `jr ra` without saving the previous return address, so **as written it is not a valid general nested-call function**. That is intentional: a careful analyst should spot it rather than treating any plausible-looking disassembly as valid.

Stop and predict

1. Why can copying a displaced `beq` verbatim into a trampoline change its behavior?
2. Why must a hook protect `ra` and ABI-preserved registers according to the call pattern?
3. What must be proved before equating a modem-view address and an AP virtual address?
4. Does a load of `0x204(s0)` prove where the flag is written? Why not?

5. Identify the hidden calling-convention flaw in the final synthetic function and describe a possible correction.

Pass standard: present an evidence chain from observed word to possible producer, and explain at least three ways a naïve patch/debugger could corrupt execution.

Appendix A. MIPS Field Reference

This appendix is meant for **lookup**, not sequential memorization. Always verify revision-specific behavior in the appropriate architectural manual [M1-M3]. The explanations below generally assume conventional classic MIPS32 semantics unless explicitly qualified.

A.1 General-purpose register map

No.	ABI name	Conventional use	Common RE clue
0	`$zero`	constant 0	comparison against zero / sink
1	`$at`	assembler temporary	macro-expanded instruction
2	`$v0`	return value 0	status/result/temp
3	`$v1`	return value 1	wider return value
4	`$a0`	argument 0	candidate pointer input
5	`$a1`	argument 1	count/length/flag
6	`$a2`	argument 2	data/size/flags
7	`$a3`	argument 3	fourth argument / ABI uses
8	`$t0`	caller-saved temp	short-lived intermediate
9	`$t1`	caller-saved temp	short-lived intermediate
10	`$t2`	caller-saved temp	loop or value
11	`$t3`	caller-saved temp	loop or value
12	`$t4`	caller-saved temp	temporary
13	`$t5`	caller-saved temp	temporary
14	`$t6`	caller-saved temp	temporary
15	`$t7`	caller-saved temp	temporary
16	`$s0`	callee-saved	long-lived object base
17	`$s1`	callee-saved	long-lived state
18	`$s2`	callee-saved	long-lived state
19	`$s3`	callee-saved	long-lived state
20	`$s4`	callee-saved	long-lived state
21	`$s5`	callee-saved	long-lived state
22	`$s6`	callee-saved	long-lived state
23	`$s7`	callee-saved	long-lived state
24	`$t8`	caller-saved temp	temporary
25	`$t9`	caller-saved / PIC	indirect-call address
26	`$k0`	kernel reserved	exception-entry scratch
27	`$k1`	kernel reserved	exception-entry scratch
28	`$gp`	global pointer	GOT/small global-data access
29	`$sp`	stack pointer	current frame
30	`$fp`/`$s8`	frame pointer/saved	frame-based addressing
31	`$ra`	return address	call/return flow

When reading code: record the **current value** separately from the **typical ABI role**. A register name alone does not prove a data type or a particular memory region.

A.2 Common load/store quick reference

Mnemonic	Semantic direction	Classic MIPS32 notes
`lb rt,o(rs)`	byte memory -> `rt`	sign extends 8 bits
`lbu rt,o(rs)`	byte memory -> `rt`	zero extends 8 bits
`lh rt,o(rs)`	2-byte memory -> `rt`	sign extends 16 bits; aligned
`lhu rt,o(rs)`	2-byte memory -> `rt`	zero extends; aligned
`lw rt,o(rs)`	4-byte memory -> `rt`	aligned word load
`sb rt,o(rs)`	low 8 `rt` bits -> memory	1 byte
`sh rt,o(rs)`	low 16 `rt` bits -> memory	2 bytes; aligned
`sw rt,o(rs)`	low 32 `rt` bits -> memory	4 bytes; aligned
`lwl/lwr`	partial word assembly	ISA/endian-specific details
`swl/swr`	partial word write	ISA/endian-specific details
`ll/sc`	load-linked/store-conditional	atomic sequences, platform constraints

A.3 Arithmetic, logical, and control-flow quick reference

Mnemonic	Core purpose	Typical investigation use
`addu`, `subu`	arithmetic without signed-overflow trap	pointer / index math
`addiu`	addition with sign-extended immediate	stack adjustment, pointer displacement
`lui`	immediate shifted into high 16 bits	32-bit address construction
`andi`, `ori`, `xori`	bitwise operations with zero-extended immediate	flags and masks
`and`, `or`, `xor`, `nor`	register-register bitwise operations	flags, bitfields
`sll`, `srl`, `sra`	fixed shifts	indexing, sign handling
`slt`, `sltu`, `slti`, `sltiu`	comparisons	signed/unsigned predicates
`beq`, `bne`	branch on equality/inequality	conditions, loops
`j`, `jr`	unconditional direct / register jump	dispatch, return
`jal`, `jalr`	jump and link	calls, indirect calls
`break`	breakpoint exception	software debugger trap
`syscall`	system-call exception	kernel service transfer
`sync`	memory ordering	synchronization
`mfc0`, `mtc0`	CP0 register transfer	privileged state management
`eret`	exception return	resumes from privileged exception

Pseudoinstruction warning: `move`, `li`, `la`, `beqz`, `bnez`, and `nop` are often presented as assembly shorthand; map to the actual encoded instructions before patching.

A.4 Critical mental formulas

```
Byte count = bit count / 8
```

```
Effective address = base register value + signed byte offset

I-type branch target (classic) = PC + 4 + (sext16(imm) << 2)

Classic delayed JAL return address = instruction_PC + 8

Little-endian word for memory bytes 78 56 34 12 = 0x12345678

Masked bit test: value & (1 << bit_index)

Relative field offset = absolute-within-region offset - subregion start
```

A.5 Common mistakes and their corrections

Incorrect belief	Correct interpretation
`0x40` is the sixth bit	`0x40` sets bit **index 6**, seventh from right
A register name identifies its object type	Register use and provenance establish meaning
`sw v0,4(s0)` changes `v0`	It writes register contents to memory
`lw v0,4(s0)` puts the address in `v0`	It reads the **word stored at** that address
`addiu` zero-extends immediate because of `u`	Immediate is sign-extended; `u` relates to overflow behavior
A matching field offset proves shared-memory access	Must prove base pointer and mapping
A branch skips the next instruction	Not generally on classic delayed-branch MIPS
Every MIPS instruction occupies four bytes	Compressed MIPS modes can differ
Writing patch bytes guarantees execution	Instruction cache and synchronization may intervene
An exception always reports exact trap PC in EPC	Classic delay-slot exceptions need BD/EPC interpretation
A Ghidra decompile is ground truth	It is an analysis hypothesis grounded in disassembly

Appendix B. Checkpoint Answer Key

Instruction to the reader: cover these answers while working. After writing an answer, compare **the reasoning** as well as the numerical result. If uncertain, invent an extra example and solve it before marking the lesson complete.

B.1 Lesson 1

1. `0x2C = 2*16 + 12 = 44` decimal, because hexadecimal position 1 has weight 16. **2.** `0x5000 + 0x204 = 0x5204`. **3.** Little-endian bytes `EF BE AD DE` form `0xDEADBEEF`. **4.** `0xFC04` is four-byte aligned; `0xFC06` is not. In hexadecimal, addresses ending in `0,4,8,C` are divisible by four. **5.** The observation shows a reproducible `0x40` difference correlated with the tested screen state; `SCREEN_OFF` as the official semantic meaning remains a hypothesis.

B.2 Lesson 2

1. Writes to architectural register `zero` are discarded; reads remain zero. **2.** `t0=0x30`; `t1` and `t2` unchanged. **3.** `lw`: memory word at `s1+12` flows into `t0`; `sw`: value in `t0` flows to memory at `s1+12`. **4.** The field displacement is only part of the effective address; `s0` may point to another object. **5.** The ISA defines things like constant-zero behavior; the ABI defines conventions such as argument registers and which registers must be preserved by a function.

B.3 Lesson 3

1. For `sw v0,8(s0)`, effective address `0x3008`; new memory word `0x12345678`; both registers unchanged. **2.** For `lw`, new `v0=0xAAAAAAAA`, memory word unchanged, `s0` unchanged. **3.** `lb` produces `0xFFFFFF80`; `lbu` produces `0x00000080`. **4.** The base is read from the **old** `v0`, before the destination write replaces its contents. **5.** Example alternatives: AP-side code writes an alias mapping; a copy routine uses another base/displacement; a byte/halfword store modifies part of the word; DMA or another processor writes the backing memory. These are possibilities, not confirmed project facts.

B.4 Lesson 4

1. `0xC1 & 0x40 = 0x40`; `0xC1 | 0x40 = 0xC1`; `0xC1 ^ 0x40 = 0x81`. **2.** Immediate `0xFFFF` in `addiu` is sign-extended to -1, so result is `0xFFFFFFFF`. **3.** `0x80 & 0x40 = 0`. **4.** With unsigned values, `0xFFFFFFFF` is greater than 1, so `sltu=0`; as signed `-1 < +1`, so `slt=1`. **5.** `andi` isolates selected bits and branch reacts to their value; no instruction names the business semantics of the bit.

B.5 Lesson 5

1. In classic delayed MIPS32, `ra=0x3008`. **2.** Yes: a normal branch's delay-slot instruction executes even when the branch is taken. **3.** The slot must hold a legal, safe instruction; `nop` avoids an unwanted side effect. **4.** `0x1000 + 4 + (3*4) = 0x1010`. **5.** It can skip the original instruction, mishandle branch/delay-slot restart state, cross an instruction-mode boundary, or continue at an incorrect control-flow target.

B.6 Lesson 6

1. `sp=0x80000FE0`. **2.** `0x80000FE0 + 0x1C = 0x80000FFC`. **3.** Restore the caller-visible original `s2` value before returning, normally by saving and later reloading it. **4.** `sp` commonly addresses a local

frame; provenance and mapping distinguish stack slots from other structures. **5.** `a0` only tells us argument position, not semantic type; inspect the caller's value construction, pointer provenance, and call signature.

B.7 Lesson 7

1. Endianness orders the **bytes of the 32-bit machine word** from least significant to most significant in little-endian memory. **2.** Opcode 6 bits, base `rs` 5 bits, target register `rt` 5 bits, immediate 16 bits. **3.** Inspect the function's callers and trace where their `a0` values came from, including global initialization. **4.** Another object may use `+0x204` (false positive); computed/indirect accesses may lack the immediate (false negative). **5.** Runtime trace, breakpoint observation, or another independent execution observation under the screen transition, with correct address mapping and provenance.

B.8 Lesson 8

1. To permit exact restoration before the displaced instruction executes; otherwise the function has lost original behavior. **2.** The increment of `t0` is skipped. **3.** On relevant classic MIPS, BD can indicate that the exception occurred in a delay slot while EPC names the branch. **4.** Prefer a supported write watchpoint because the question is who writes that data; follow-up must still resolve mapping/aliasing and target capabilities. **5.** The instruction fetch side may still see the old cached instruction; a different mapping, CPU core, or permission may prevent execution of patched bytes.

B.9 Lesson 9

1. The branch displacement is computed relative to the branch instruction's own PC, so relocation changes the target unless adjusted; delay-slot context also matters. **2.** Hook code and calls may overwrite `ra`, `a`/`t` registers, and preserved `s` state; restore what the interrupted code expects, respecting ABI and full execution context. **3.** Actual page tables, remapping descriptors, or other platform evidence must show that both addresses refer to the same backing storage. **4.** `lw` reads; determining who writes requires store-side or runtime writer evidence. **5.** `jal notify_helper` overwrites `ra`; before `jr ra`, the function must preserve its caller's return address (for example, in a properly allocated stack frame) and restore it. As written it is an intentionally faulty sketch.

Appendix C. Advanced MIPS Topics You Will Encounter Later

The four-hour-and-fifty-minute schedule is an **introduction to necessary capabilities**. A professional debugger on a specific processor may require detailed knowledge beyond the lesson durations. This appendix tells you **what those topics are and when they become relevant**, so they are not silently omitted.

C.1 HI/LO multiplication, division, and hazards

Classic MIPS has special HI/LO registers used by `mult/multu/div/divu` and accessed via `mfhi/mflo`. For a full-width multiplication result, LO contains lower result bits while HI contains upper bits; division uses these for quotient/remainder according to instruction definitions. Instructions and hazards vary across ISA revisions and implementations. When stopping a thread inside an arithmetic sequence, consider whether a debugger must save this state in addition to general-purpose registers. Never infer a divide-by-zero result from mathematical convention without consulting the ISA.

C.2 Atomic `ll/sc` and breakpoint safety

`ll` (load-linked) and `sc` (store-conditional) support atomic retry loops on implementations that provide them. Conceptually, `ll` reads while establishing a reservation and `sc` succeeds only if reservation conditions are met; its destination register reports success/failure according to ISA rules. Reservation granularity, context-switch effects, interrupts, and coherence vary. A debugger that inserts traps between linked/conditional operations may change the success rate or force repeated retries. Debugging atomic code thus requires understanding not only the instructions but also reservation invalidation and multicore behavior.

C.3 COP1 and floating-point state

On targets implementing floating point, COP1 instructions operate on floating-point registers and status/control state. Floating-point argument passing can differ from integer O32 conventions; 32-bit and 64-bit FPR organization depends on CPU configuration and ABI. Debugger correctness may require saving FPRs and floating-point control/status, and GDB needs a matching target register description. An integer-only stub cannot claim to capture the full machine context if the firmware uses floating-point state.

C.4 CP0 configuration and architectural hazards

CP0 controls exception handling, privileges, translations, timers, and implementation-defined features. Writes to privileged registers can require **hazard barriers** before later instructions observe new state. MIPS revisions define instructions such as `ehb` and `eret` with specific semantics; do not insert them without confirming support and required sequence. A debugger should minimally disturb CP0 state and should record precisely which bits it changes. Interrupt enable/disable transitions can affect real-time firmware, so “safe” handling cannot be assumed.

C.5 TLB, virtual segments, and fault analysis

A TLB provides address translations for mapped regions; unmapped segments on certain classic MIPS32 privilege configurations bypass normal TLB translation. Addressing categories differ

across MIPS32/MIPS64 and revisions. TLB entries can encode page frames, address-space identifiers, validity, dirty/write state, and global attributes. Relevant faults include TLB refill/miss and TLB invalid/modification cases. A debugger interpreting a fault must inspect the actual exception code, fault address, privilege state, and current translation configuration. Do not generalize an address such as `0x80000000` without identifying the core's addressing mode.

C.6 microMIPS, MIPS16e, and instruction patching

Compressed modes can use 16-bit and mixed-width instruction encodings. A debugger that universally saves/restores four bytes at the current PC may split an instruction. Branch displacement, call mode transitions, breakpoint encoding, disassembly alignment, and exception reporting can differ. **ISA mode identification is a precondition**, not an optional optimization. The final debugger should maintain explicit per-location instruction length and decoding mode.

C.7 DSP, SIMD, and implementation-specific extensions

MIPS DSP and MSA provide additional operations/register state on supported cores. Firmware for multimedia or communication processors may also include vendor-specific extensions or coprocessors. Decompiler interpretations can degrade when an extension is unsupported; custom instruction language definitions or disassembler plugins may be needed. Before claiming a complete context save, inventory every execution-state facility your firmware can use.

C.8 Concurrency and interrupt-time races

A hook running in an interrupt handler has different safety requirements from a hook in ordinary thread context. Reentrancy, locks, log buffers, stack availability, nonblocking communication, and memory ordering all matter. A breakpoint temporarily removed to single-step on one CPU can be missed by another CPU unless the system is synchronized or uses per-core hardware facilities. Validate designs first on isolated, controlled routines and make every test reversible.

C.9 Firmware boot, relocation, and runtime memory identity

A ROM image may execute directly from a mapped range, be copied to RAM, be decompressed, or have sections relocated at boot. Raw file offsets may therefore not match runtime addresses. A decompiler's constant may represent a link-time value before remapping. Before patching or debugging, determine the **actual runtime instruction address**, the backing storage of code/data, and how any instruction/data caches relate to that mapping. This is especially important when source refers to separate AP/modem memory views.

Appendix D. Debugger Correctness Checklist

Use this as a preflight **design checklist**, not a recipe to modify unknown firmware. Each box should be supported by observed behavior, documentation, or a controlled test.

D.1 Architecture identity

- ☐ ISA family, revision, and current instruction mode identified.
- ☐ Register width, endianness, and calling convention identified.
- ☐ Exceptions, delay-slot behavior, CP0 features, and hardware-debug support verified.
- ☐ Instruction lengths and patch boundaries are known for the exact location.

D.2 Memory model

- ☐ Runtime virtual and physical mappings are understood.
- ☐ RAM, ROM, MMIO, and shared-memory regions are distinguished.
- ☐ Permissions are known for reads, writes, and execution.
- ☐ Cache maintenance and multicore synchronization are defined for patches.
- ☐ Debug reads avoid destructive device side effects where applicable.

D.3 Software breakpoint invariants

- ☐ Saved original instruction bytes are correct.
- ☐ Breakpoint encoding is valid for location and ISA mode.
- ☐ Patched instruction is visible to instruction fetch.
- ☐ Trapped execution context is captured before handler clobbering.
- ☐ EPC and branch-delay state interpreted correctly.
- ☐ Original instruction executes exactly once on resume.
- ☐ Breakpoint is reinstalled without races or infinite retriggering.
- ☐ Removing a breakpoint restores the exact original program bytes.

D.4 Context and protocol invariants

- ☐ All required general, special, privileged, and optional processor state is accounted for.
- ☐ Host register numbering/widths match the actual target description.
- ☐ Continue and step requests have verified semantics, including branches.
- ☐ Stop reasons and errors are reported unambiguously.
- ☐ Communication retries, checksums, and disconnect recovery are understood.

D.5 Instrumentation invariants

- ☐ Relocated instructions preserve PC-dependent semantics.
- ☐ Delay-slot relationships are preserved.
- ☐ Scratch and ABI-preserved registers are handled correctly.
- ☐ Hook stack use and interrupt/reentrancy constraints are safe.
- ☐ Patch is reversible, tested, and monitored for faults.

D.6 Suggested validation tests

Test 1: Register preservation. Initialize every relevant GPR to a distinctive sentinel in a controllable test routine. Trigger and resume. Confirm unchanged registers remain exact. **Test 2: Side-effect count.** Replace a known increment instruction with a breakpoint; after continuing once, the counter increments **once**, not zero or twice. **Test 3: Branch path.** Exercise both taken and not-taken branches with breakpoints in safe positions; check delay-slot effects. **Test 4: Cache visibility.** Confirm both data reads and actual instruction fetch observe intended patch states. **Test 5: Multiple cores.** If present, verify stop/resume and breakpoint reinstallation do not produce silent races. **Test 6: Removal.** Restore original bytes and show the code runs identically to the uninstrumented baseline under controlled inputs.

Appendix E. Worked Integrative Case Study

This case study combines the nine lessons. It is explicitly **synthetic** and exists to practice reasoning. It is **not** a recovered MediaTek function.

E.1 The code and starting state

Assume classic little-endian MIPS32 Release 2 with O32-style conventions. `a0` points to a readable `struct`, `a1` is a bit mask, `sp=0x80003000`, and incoming `ra=0x80001180`. A status word at `a0+0x204` is `0x00000040`.

```
flag_test:
    addiu sp, sp, -24
    sw    ra, 20(sp)
    sw    s0, 16(sp)
    move  s0, a0
    lw    t0, 0x204(s0)
    and   t0, t0, a1
    sltu  v0, zero, t0
    lw    s0, 16(sp)
    lw    ra, 20(sp)
    jr    ra
    addiu sp, sp, 24
```

Let `a0=0x4000FA00` and `a1=0x40`. Work out every address and register value; distinguish the **new value of `v0`** from the **raw bit mask**. The function is a leaf but uses a pedagogical frame to show saves/restores.

E.2 Trace the frame

After `addiu sp,sp,-24`, `sp=0x80002FE8`. `ra` is stored at `0x80002FFC`. `s0` is stored at `0x80002FF8`. `move s0,a0` gives `s0=0x4000FA00`. The load uses effective address `0x4000FC04` and reads the assumed word `0x40` into `t0`. The AND with `a1=0x40` leaves `0x40`. `sltu v0,zero,t0` sets `v0=1` because unsigned zero is less than 0x40. Saved `s0` and `ra` are restored. The delay-slot stack increment restores `sp=0x80003000` before execution returns to `0x80001180`.

The function's result is **1**, not `0x40`, because `sltu` converts the nonzero mask to a canonical boolean integer. If the status word were zero, result would be zero. This distinction appears frequently in compiled code.

E.3 Proving identity is a separate proof obligation

The synthetic setup stipulated that `a0=0x4000FA00`. In a real image, you would have to **prove** this through call-site arguments, initialization logic, and mapping. Even if proven, a read says only this function consumes the field. If the objective is to find the writer, identify stores to the backing location through AP/modem aliases or a supported watchpoint, then validate execution timing.

E.4 Make a prediction under a changed state

Suppose incoming `a1=0x20` while the status word is still `0x40`. Then `0x40 & 0x20=0`, so result `v0=0`. The same memory field can yield different returned booleans depending on the **mask argument**. This is why tracing both pointer provenance and argument values matters.

E.5 Your capstone exercise

Without using the answer above, repeat the trace for `a0=0x4000FA00`, `a1=0x40`, memory word `0x000000C0`, initial `sp=0x80003000`. Then identify: (a) every memory write performed by the function; (b) every memory read; (c) which source-level statement the mask operation might represent; (d) two falsifiable hypotheses about the observed field; and (e) the next cheapest experiment to distinguish the hypotheses. Your answer is complete only when you can defend each step using instruction semantics and evidence.

Appendix F. Glossary

ABI: Calling convention and binary interface rules for functions, registers, stack layout, and data representation. **Address:** Numeric location used to select memory. **Alignment:** Required divisibility of an address for a memory width. **AP:** Application processor, the host-side processor in the context notes. **BD:** Classic MIPS exception-state indication related to a branch delay slot. **Bitmask:** Value used to select or modify particular bit positions. **Breakpoint:** Mechanism for stopping execution at a location or condition. **Byte:** Eight bits. **Cache:** Small storage that may hold instruction or data copies. **Callee-saved:** Register state a called function must preserve according to its ABI. **Calling convention:** Agreed rules for parameters, return values, saved registers, and stack. **CP0:** Privileged MIPS processor control/exception facilities. **Delay slot:** Instruction position associated with a delayed control transfer on relevant MIPS ISAs. **Dereference:** Read/write memory addressed by a pointer. **Detour:** Redirection of original execution into replacement or instrumentation code. **Effective address:** Address calculated for a memory access. **Endianness:** Byte order of a multi-byte value. **EPC:** Exception program counter for relevant restart semantics. **Exception:** Transfer to privileged handler caused by a condition such as breakpoint or address fault. **GPR:** General-purpose register. **HI/LO:** Classic arithmetic-result special registers. **Hook:** Instrumentation or redirected behavior inserted at a program point. **ISA:** Instruction Set Architecture, the contract defining machine instructions and architectural behavior. **MMIO:** Memory-mapped I/O, device registers accessed using addresses. **Offset:** Displacement from a base. **PC:** Program counter or execution-location state. **Pseudoinstruction:** Assembly shorthand expanded to real machine instructions. **Register:** CPU-visible state slot referenced by instructions. **Shared memory:** Backing memory accessible from multiple execution agents/views. **Sign extension:** Widening that propagates a signed value’s top bit. **Stack frame:** Function-reserved region for saves, locals, and arguments. **Store:** Writes register-derived data to memory. **TLB:** Translation lookaside buffer, a cache of address translations. **Trampoline:** Code used to execute relocated instructions and/or reconnect a detoured flow. **Watchpoint:** Mechanism that detects selected data accesses. **Word:** In this book’s MIPS32 examples, 32 bits. **XREF:** Static reference connecting instructions/data to another location. **Zero extension:** Widening that fills high bits with zeros.

Appendix G. Five Detailed Tracing Laboratories

These laboratories are for **independent practice after the scheduled session**. All code and values are synthetic. Use paper to write the register state after each line before checking the walkthrough. These examples deliberately include traps common in reverse engineering.

G.1 The register stops being a pointer

```
# Initial: a0=0x80001000
# Memory byte [0x8000100E]=0xC0
move s0, a0
lw   v0, 4(s0)        # memory word [0x80001004]=0x80002000
lbu  v0, 0xE(v0)      # memory byte [0x8000200E]=0xC0
andi v0, v0, 0x40
```

Trace: `s0=0x80001000`. The first `lw` loads the **word stored at** `0x80001004`, which the setup says is `0x80002000`. At that point `v0` contains a **pointer**. The `lbu` uses the **old** pointer to access `0x8000200E` and replaces `v0` with `0x000000C0`. The `andi` produces `0x00000040`. The final `v0` is no longer a pointer. Merely following the textual register name `v0` without tracking definitions is a mistake.

Analysis question: if that first loaded pointer could vary among calls, what additional proof is needed to identify the target object? Answer: trace where the word at `s0+4` is initialized or updated, and determine its possible run-time values, not just its current example value.

G.2 Signed load changes a branch

```
# Memory byte [a0+3]=0xF0
a0 points to a valid object
lb   t0, 3(a0)
slti t1, t0, 0
bne  t1, zero, negative
nop
```

The second line is a **statement of the initial condition**, not assembly. `lb` reads `0xF0` and sign-extends it to `0xFFFFFFF0` (signed -16). `slti` compares -16 with 0 as signed integers and sets `t1=1`. The branch is taken; the delay-slot `nop` still executes on classic MIPS. If the load were `lbu`, `t0=0x000000F0` (240), so `slti` would produce 0 and the branch would not be taken. **One letter in a load mnemonic changes program decisions.**

G.3 Reconstruct a cumulative shared-memory layout

Suppose a *synthetic* region table has these entries, each with an initial zero offset:

Name	Size in hex	Runtime offset
A	`0x2000`	`0x0000`
B	`0x5000`	`0x2000`
C	`0x3000`	`0x7000`
D	`0x0900`	`0xA000`

The initialization loop assigns each new start to `previous.offset + previous.size`, assuming no extra alignment or gaps. To establish the interval containing a target offset `0xA204`, compare bounds: A covers `[0,0x2000)`, B `[0x2000,0x7000)`, C `[0x7000,0xA000)`, D `[0xA000,0xA900)`. The target is in D, at **relative offset `0x204`** because `0xA204-0xA000=0x204`.

This proof uses **interval membership**, not mere proximity. If any entry is rounded up to an alignment boundary, or if runtime sizes differ from static initializers, recompute the table. The method directly mirrors the reasoning needed for the actual `CCISM_SCP` investigation but uses invented values and names.

G.4 Track argument preparation across a call

```
# Caller snippet; classic MIPS32
lui   a0, 0x4000
ori   a0, a0, 0xFA00
jal   process_region
nop

process_region:
    addiu sp, sp, -24
    sw    ra, 20(sp)
    sw    s0, 16(sp)
    move  s0, a0
    lw    v0, 0x204(s0)
    lw    s0, 16(sp)
    lw    ra, 20(sp)
    jr    ra
    addiu sp, sp, 24
```

The caller prepares `a0=0x4000FA00`. `jal` transfers control after its delay slot and puts the correct return address in `ra`. The callee uses its incoming `a0` as the base; the load's effective address is `0x4000FC04`. Within this **constructed program**, it is proved that the callee reads from that address. In real firmware, you must still prove that the code executes and that the virtual/modem mapping points at the intended backing memory.

This example also explains why **tracing only a callee is insufficient**: a function parameter is a dependency supplied by the caller. If multiple callers provide different `a0` values, the same `0x204` load can access multiple objects.

G.5 Software-breakpoint execution-once problem

Original code:

```
X:      addiu t0,t0,1
X+4:    andi  t1,t0,0xFF
```

The breakpoint manager saves the original 32-bit instruction at `X` and patches an appropriate `break` instruction. The CPU traps at `X`; its handler captures the execution context. Two naïve resume algorithms fail in different ways.

Failure A, resume at X: because `break` remains present, the CPU traps again immediately; no progress. **Failure B, resume at X+4:** because the increment is skipped, `t0` is stale; subsequent code behaves differently. A sound approach restores the original and arranges for its effect to

execute exactly once, then reinstalls the breakpoint without allowing an unintended core to slip through. The correct mechanism depends on hardware single-step support, branch/delay-slot behavior, per-core coordination, and privileges. A *temporary breakpoint at the next sequential address* is insufficient if the displaced instruction is a branch or jump; after it executes, the next architectural instruction may be elsewhere.

Verifier's question: If `t0` begins as 10 and there is one hit, what is the value when continuing past the original increment? Correct answer: 11. Zero increments or two increments means the breakpoint resume protocol is wrong.

Appendix H. Target-Identification Worksheet

Fill this **before transferring instruction or debugger examples to a real image**. Do not infer unknown fields from the fact that an instruction *looks* familiar.

Question	Evidence to collect	Current status
Which processor executes the analyzed image?	platform docs, boot logs, header metadata, traces	Unknown until established
ISA width and revision?	vendor docs, ELF flags, verified opcode features	Unknown until established
Instruction mode(s)?	decode patterns, mode transitions, target docs	Unknown until established
Byte order?	ELF data encoding, known test words	Unknown until established
ABI convention?	binary metadata, prologues/callers, toolchain	Unknown until established
Code runtime base?	loader logic, relocations, memory map	Unknown until established
CP0/exception features?	architecture implementation docs, tests	Unknown until established
Hardware debug facilities?	implementation manuals/probes	Unknown until established
Are code pages writable/executable?	permissions and actual controlled tests	Unknown until established
How is code-cache visibility maintained?	core/platform cache maintenance documentation	Unknown until established
Where is shared backing RAM?	reservation source, remap registers, mappings	Partial hypothesis from project notes
Which processor produces the screen-correlated field?	stores/watchpoints/source evidence	Not established

This worksheet deliberately contains **Unknown** entries. Replacing those with invented values would make the debugger design look more complete while actually weakening it.

References and Further Reading

Technical references. The numbered source codes below are used throughout the book. The precise *instruction behavior* for any real processor must be checked against its architecture revision and vendor documentation. Some sources are collected through Cornell University's publicly available teaching resource index; others are tool manuals or ABI documentation.

[M1] MIPS Technologies, *MIPS32 Architecture for Programmers, Volume I-A: Introduction to the MIPS32 Architecture*. Public university resource index: https://www.csl.cornell.edu/~cbatten/ece4750/resources.html.

[M2] MIPS Technologies, *MIPS32 Architecture for Programmers, Volume II-A: The MIPS32 Instruction Set*. Same university resource index: https://www.csl.cornell.edu/~cbatten/ece4750/resources.html. Primary authority for instruction encoding and semantics, with revision-specific details.

[M3] MIPS Technologies, *MIPS32 Architecture for Programmers, Volume III: MIPS32 and microMIPS32 Privileged Resource Architecture*. Same resource index. Primary authority for exception state, CP0, TLB, address translation, and privilege features.

[M4] dyncall project, *MIPS O32 Calling Convention* and related ABI convention tables: https://dyncall.org/docs/manual/manualse11.html. For production work, reconcile against the ABI document and compiler/toolchain actually used by the target.

[T1] Ghidra, *References* documentation: https://www.ghidradocs.com/9.1_PUBLIC/help/Base/help/topics/ReferencesPlugin/References.htm. Explains forward and back references and their analysis role.

[T2] Ghidra, *P-Code Reference Manual*: https://ghidra.re/ghidra_docs/languages/html/pcoderef.html. Explains the intermediate operations underlying data-flow analysis and decompilation.

[T3] GNU GDB manual, *Remote Serial Protocol*. An accessible overview and packet discussion: https://developer.apple.com/library/archive/documentation/DeveloperTools/gdb/gdb/gdb_33.html. Consult the maintained GDB documentation for the version actually used.

[E1] Barak Rosenshine (2012), *Principles of Instruction: Research-Based Strategies That All Teachers Should Know*, *American Educator*. https://www.aft.org/ae/spring2012/rosenshine. Supports the lesson structure of small steps, modeling, worked examples, and guided practice.

[E2] John Dunlosky and colleagues (2013), *Improving Students' Learning With Effective Learning Techniques*, *Psychological Science in the Public Interest*, 14(1), 4-58. DOI: https://doi.org/10.1177/1529100612453266. Practice testing and distributed practice are especially useful for durable retention.

Recommended deep texts (supplementary): David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface* (MIPS-oriented editions); Dominic Sweetman, *See MIPS Run*, second edition; Randal E. Bryant and David R. O'Hallaron, *Computer Systems: A Programmer's Perspective* (for general systems reasoning; not a MIPS ISA manual). Editions matter, and they should not override the actual target's ISA and vendor documentation.

Final reminder: what this book can and cannot certify

This book is a structured companion, not a substitute for the official ISA manuals or for tested, independent practice. Prove instruction effects, pointer provenance, execution paths, and the context required for correct breakpoint recovery.